BÁO CÁO TIẾN ĐỘ 2

SPULTI-CLOUD

Học phần: Điện toán đám mây

Nền tảng thuê máy spot đa đám mây(AWS và Google Cloud)

Nhóm thực hiện: nhóm số 9
Thành viên:
Vũ Trí Sơn - 24021616
Lê Thành Long - 24021552
Lại Tùng Lâm - 24021544

TỔNG HỢP THAY ĐỔI

Chương 2:

Bổ sung chi tiết các thành phần kiến trúc.

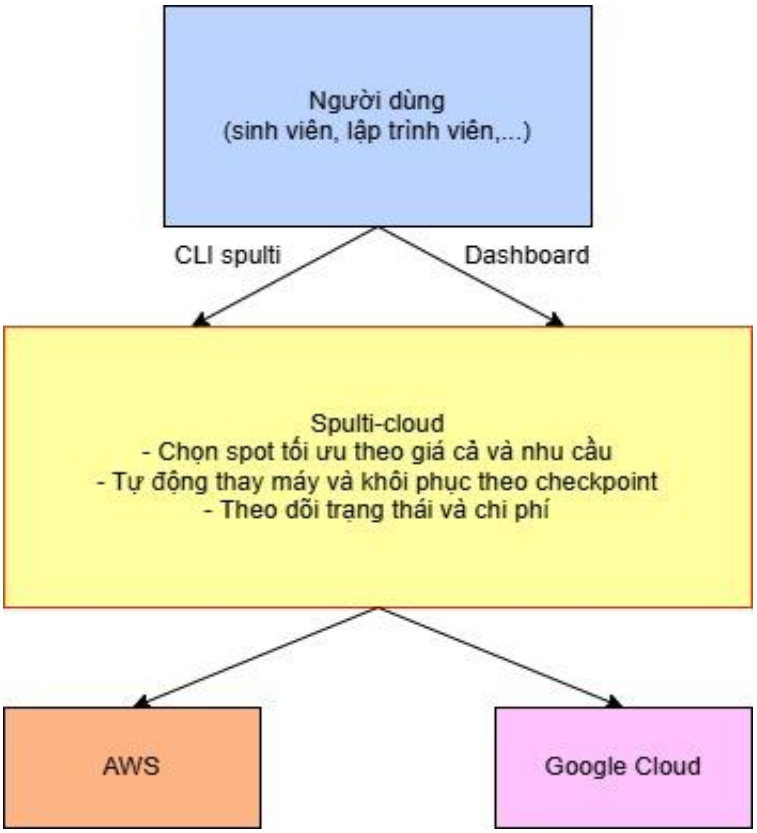
Chương 3:

Bổ sung bảng tổng hợp tiến độ công việc hiện tại.

CHƯƠNG 1: TỔNG QUAN VỀ DỰ ÁN

1.1 Giới thiệu chung

Máy spot (EC2 Spot Instances của AWS, Spot VMs của Google Cloud) là phần năng lực tính toán đang dư thừa mà nhà cung cấp đám mây bán lại với giá thấp hơn nhiều so với máy on-demand, thường rẻ hơn khoảng 60–90%. Đổi lại, nhà cung cấp có quyền thu hồi máy bất kỳ lúc nào khi cần lấy lại tài nguyên và chỉ báo trước rất ngắn. Giá và mức độ thường xuyên bị thu hồi thay đổi theo loại máy, vùng (region) và thời điểm; mỗi nhà cung cấp lại có giao diện, API và cách tính giá riêng. Vì vậy, một người dùng cá nhân muốn tận dụng máy spot phải tự so sánh giá ở nhiều nơi, tự theo dõi máy và tự xử lý khi máy bị thu hồi, vừa tốn công vừa dễ mất kết quả tính toán.



Hình 1.1. Tổng quan về Spulti-cloud

Spulti-cloud là nền tảng thuê máy spot đa đám mây nhằm giải quyết vấn đề trên. Các tính năng chính của **Spulti-cloud** gồm:

- **Thuê máy theo cấu hình trừu tượng:** người dùng mô tả nhu cầu tài nguyên, không phụ thuộc tên loại máy hay nhà cung cấp.
- **Chọn máy tối ưu trên nhiều đám mây:** so sánh đồng thời giá và rủi ro thu hồi của AWS và GCP theo ba chế độ **Tiết kiệm, Ổn định, Cân bằng**.
- **Tự động phục hồi khi bị thu hồi:** chuyển công việc sang máy mới, khôi phục checkpoint và chạy lại lệnh huấn luyện mà người dùng không phải can thiệp.
- **Truy cập và quản lý thuận tiện:** làm việc trên máy qua SSH; quản lý máy qua công cụ dòng lệnh (CLI) spulti hoặc Web Dashboard.
- **Minh bạch chi phí:** thống kê giờ chạy, số lần bị thu hồi, chi phí thực tế và phần trăm tiết kiệm so với on-demand.

Về mô hình vận hành, Spulti-cloud sử dụng tài khoản AWS và Google Cloud của chính nền tảng (do nhóm phát triển quản lý) để tạo máy, sau đó cấp quyền truy cập cho người dùng. Cách làm này giúp người dùng không cần có tài khoản đám mây hay tự cấu hình phân quyền, đồng thời giúp nền tảng gom dữ liệu giá và lịch sử thu hồi về một nơi để ra quyết định tốt hơn. Trong phạm vi dự án, nền tảng chỉ ghi nhận chi phí quy đổi cho từng người dùng, chưa tích hợp thanh toán.

1.2. Người dùng của dự án

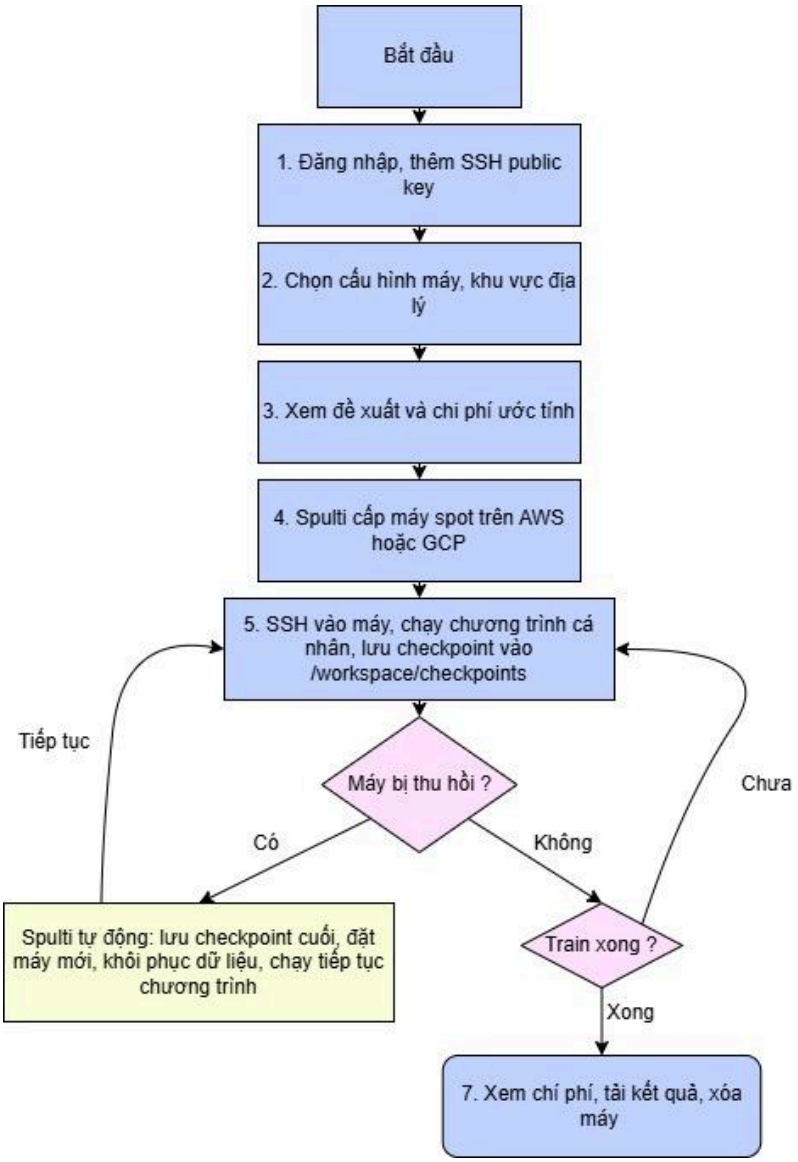
Đối tượng người dùng chính của Spulti-cloud là những cá nhân có nhu cầu huấn luyện mô hình học máy, bao gồm sinh viên thực hiện bài tập lớn, đồ án; học viên, nghiên cứu viên chạy thực nghiệm và lập trình viên cá nhân. Nhóm người dùng này có một số đặc điểm chung khiến Spulti-cloud phù hợp:

- Nhu cầu tiết kiệm chi phí.
- Công việc huấn luyện chạy dài cần checkpoint để tránh mất tiến độ, chịu được gián đoạn, việc khôi phục được tự động hóa.
- Không chuyên về hạ tầng đám mây và không có thời gian theo dõi máy liên tục.
- Quen làm việc với terminal, SSH và Python.

Bên cạnh người dùng cuối, hệ thống có vai trò **quản trị viên nền tảng** (nhóm phát triển): quản lý tài khoản đám mây, hạn mức (quota) máy spot, hạn mức số máy của mỗi người dùng và theo dõi tổng chi phí.

1.3. Cách thức và quy trình sử dụng sản phẩm

Người dùng tương tác với Spulti-cloud qua hai giao diện chính: công cụ dòng lệnh (CLI) spulti, phù hợp với thao tác trên terminal và tự động hóa bằng script; và Web Dashboard, hỗ trợ quản lý máy, theo dõi trạng thái, lịch sử thu hồi và chi phí. Hình 1.2 tóm tắt toàn bộ quy trình từ khi đăng nhập đến khi hoàn thành công việc; các mục bên dưới trình bày chi tiết từng quy trình.



Hình 1.2. Quy trình sử dụng Spulti-cloud để huấn luyện mô hình

1.3.1. Quy trình 1: Đăng ký và chuẩn bị

Người dùng đăng ký tài khoản trên Web Dashboard, cài đặt CLI, đăng nhập và thêm khóa SSH công khai (public key). Khóa này được nền tảng tự động cài vào mọi máy cấp cho người dùng, kể cả các máy thay thế sau này, nên người dùng chỉ cần làm bước này một lần.

1.3.2. Quy trình 2: Thuê máy

Bước 1 – Chọn cấu hình. Người dùng khai báo số vCPU, RAM, GPU (loại và số lượng, không bắt buộc), dung lượng ổ đĩa, khu vực và chế độ tối ưu. Khu vực được chọn theo nhóm địa lý; Spulti-cloud tự ánh xạ sang vùng cụ thể của từng nhà cung cấp, chẳng hạn ap-southest-1 của AWS và asia-southeast1 của GCP (đều đặt tại Singapore). Hệ điều hành mặc định là Ubuntu 22.04.

Bước 2 – Xem đề xuất. Hệ thống trả về danh sách máy phù hợp, được xếp hạng theo chế độ tối ưu đã chọn, kèm thông tin về nhà cung cấp, vùng, loại máy, giá theo giờ, rủi ro thu hồi và mức tiết kiệm so với on-demand (Bảng 1.2). Người dùng có thể xem danh sách hoặc bỏ qua để hệ thống tự chọn máy đứng đầu.

Bảng 1.2. Ví dụ kết quả đề xuất ở chế độ Cân bằng (số liệu minh họa)

Hạng	Nhà cung cấp	Vùng	Loại máy	Giá spot (USD/giờ)	Rủi ro thu hồi	Tiết kiệm
1	GCP	asia-southeast1	e2-standard-4	0,042	Trung bình	68%
2	AWS	ap-southeast-1	m6i.xlarge	0,071	Thấp	64%
3	AWS	ap-southeast-1	m5.xlarge	0,065	Trung bình	66%

Bước 3 – Tạo máy. Người dùng xác nhận tạo máy, đặt tên và tùy chọn khai báo lệnh job (--job) để tự động chạy khi máy khởi động hoặc được thay thế. Máy lần lượt chuyển qua các trạng thái PENDING → PROVISIONING → RUNNING.

1.3.3. Quy trình 3: Sử dụng máy để huấn luyện mô hình

Người dùng truy cập máy qua spulti ssh <tên-máy>. CLI mà không cần biết địa chỉ IP hiện tại, kể cả khi máy đã được thay thế. Sau khi kết nối, người dùng tải mã nguồn, dữ liệu vào /workspace,và chạy script huấn luyện. Node Agent định kỳ đồng

bộ checkpoint từ /workspace/checkpoints lên kho lưu trữ (ví dụ 5 phút/lần). Nhật ký huấn luyện có thể được xem trực tiếp bằng spulti logs mà không cần SSH.

1.3.4. Quy trình 4: Khi máy bị thu hồi (tự động)

Đây là quy trình nền tảng thực hiện hoàn toàn tự động; người dùng chỉ quan sát kết quả. Trình tự xử lý như sau:

1. Nhà cung cấp gửi cảnh báo thu hồi tới máy (trước khoảng 2 phút với AWS, khoảng 30 giây với GCP).
2. Node Agent phát hiện cảnh báo, đồng bộ checkpoint lần cuối và báo sự kiện về nền tảng.
3. Spulti-cloud chọn máy thay thế đáp ứng cùng yêu cầu cấu hình và chế độ tối ưu, tạm loại trừ nhóm máy vừa bị thu hồi để hạn chế gián đoạn lặp lại. Máy mới có thể thuộc nhà cung cấp khác.
4. Máy mới khởi động, Node Agent tải checkpoint về /workspace/checkpoints và chạy lại lệnh job; việc huấn luyện tiếp tục từ checkpoint gần nhất.
5. Người dùng nhận thông báo trên Dashboard và xem được chi tiết bằng spulti events: thời điểm thu hồi, máy mới, thời gian gián đoạn.

Chỉ dữ liệu trong thư mục checkpoint và các thư mục được đăng ký đồng bộ mới được bảo toàn khi thay máy. Mã nguồn và dataset cần được cấu hình để có thể tải lại tự động.

1.3.5. Quy trình 5: Theo dõi chi phí và kết thúc

Sau khi hoàn thành công việc, người dùng xem báo cáo gồm tổng thời gian chạy, số lần thu hồi, chi phí thực tế, chi phí on-demand tương đương và phần trăm tiết kiệm. Người dùng có thể tải kết quả về và xóa máy để giải phóng tài nguyên. Checkpoint được lưu thêm một thời gian ngắn trước khi bị xóa.

Web Dashboard hỗ trợ các thao tác trên qua ba màn hình chính: Tạo máy (cấu hình và đề xuất), Danh sách máy (trạng thái, IP, nhà cung cấp) và Chi tiết máy (lịch sử sự kiện, biểu đồ chi phí).

CHƯƠNG 2: KẾ HOẠCH THỰC HIỆN

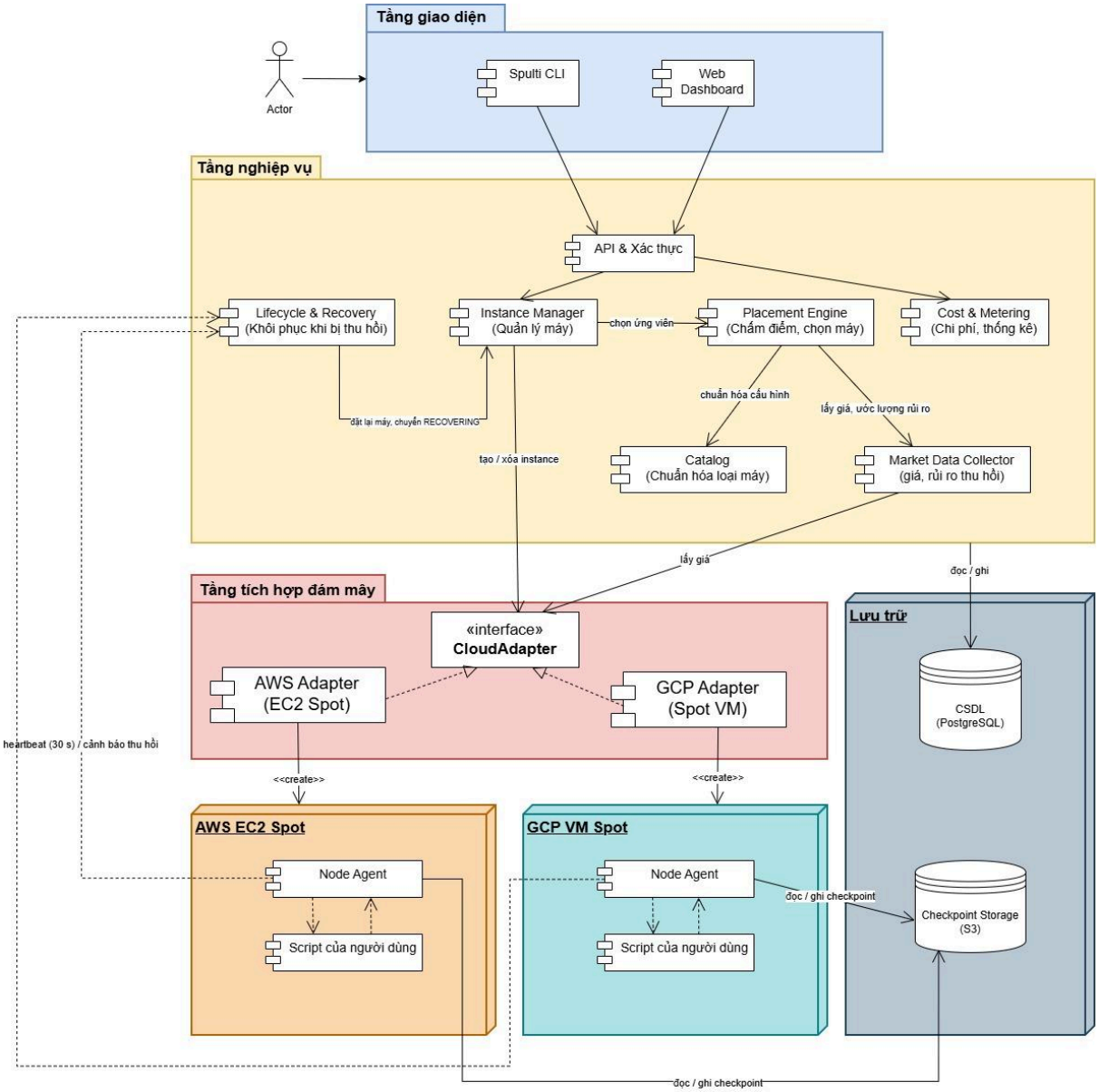
2.1 Kiến trúc về tính năng của hệ thống

Về mặt chức năng, Spulti-cloud được chia thành năm nhóm thành phần:

- Tầng giao diện
- Tầng nghiệp vụ.

- Tầng tích hợp đám mây.
- Node Agent chạy trên mỗi máy spot.
- Tầng lưu trữ.

Ý tưởng cốt lõi của kiến trúc là tách **máy Spulti** (máy logic mà người dùng nhìn thấy, có tên và cấu hình cố định) khỏi **instance** (máy vật lý cụ thể trên AWS hoặc GCP). Một máy Spulti có thể lần lượt gắn với nhiều instance theo thời gian, mỗi lần thu hồi sinh ra một instance mới; nhờ vậy người dùng không phải quan tâm máy của mình đang nằm ở đâu.



Hình 2.1. Kiến trúc chức năng của Spulti-cloud

Tầng giao diện gồm CLI spulti và Web Dashboard. Cả hai chỉ gọi cùng một REST API của control plane, nên toàn bộ logic nghiệp vụ được tập trung một chỗ và hai giao diện luôn nhất quán.

API & Xác thực tiếp nhận mọi yêu cầu: đăng ký, đăng nhập, cấp token (JWT cho Dashboard, API key cho CLI), kiểm tra quyền sở hữu máy, quản lý khóa SSH và kiểm tra hạn mức số máy của từng người dùng.

Instance Manager quản lý vòng đời của máy Spulti qua các trạng thái PENDING, PROVISIONING, RUNNING, RECOVERING, DELETED, và ghi nhận instance nào đang gắn với máy. Khi cần tạo máy, thành phần này hỏi Placement Engine để chọn ứng viên rồi giao cho Cloud Adapter tương ứng thực hiện.

Catalog chuẩn hóa các loại máy của hai nhà cung cấp (instance type của AWS, machine type của GCP) về cùng bộ thuộc tính: vCPU, RAM, GPU, kiến trúc CPU; đồng thời ánh xạ nhóm khu vực sang vùng và zone cụ thể, và ánh xạ hệ điều hành sang AMI, image của mỗi nhà cung cấp.

Market Data Collector định kỳ (ví dụ 15 phút một lần) thu thập dữ liệu thị trường: giá spot của AWS theo từng zone (API DescribeSpotPriceHistory), dữ liệu tần suất gián đoạn của AWS Spot Instance Advisor và điểm Spot Placement Score; giá Spot VM của GCP qua Cloud Billing Catalog API. Do GCP không công bố tỉ lệ thu hồi, rủi ro của máy GCP được ước lượng từ lịch sử thu hồi mà chính Spulti-cloud ghi nhận (bắt đầu từ một giá trị mặc định), và lịch sử này cũng được dùng để hiệu chỉnh số liệu của AWS.

Placement Engine là thành phần ra quyết định. Với mỗi yêu cầu, nó (1) lọc các ứng viên (nhà cung cấp, vùng, loại máy) có cấu hình lớn hơn hoặc bằng yêu cầu, đúng khu vực, còn quota và không nằm trong danh sách tạm loại trừ; (2) chuẩn hóa giá P và rủi ro R của từng ứng viên về đoạn [0, 1]; (3) tính điểm theo công thức sau và chọn ứng viên có điểm nhỏ nhất:

$$S = w \times P + (1 - w) \times R$$

trong đó w = 1 ở chế độ Tiết kiệm, w = 0 ở chế độ Ổn định và w = 0,5 ở chế độ Cân bằng; chế độ Tiết kiệm vẫn loại bỏ các ứng viên có rủi ro quá cao. Nếu nhà cung cấp báo hết dung lượng khi tạo máy, Placement Engine chuyển sang ứng viên kế tiếp trong danh sách.

Tầng tích hợp đám mây (Cloud Adapter) định nghĩa một giao diện chung (tạo, xóa, truy vấn, …) với hai cài đặt. AWS Adapter dùng thư viện boto3, tạo máy

bằng RunInstances với tùy chọn thị trường spot và truyền script khởi tạo (user-data) để cài NodeAgent. GCP Adapter dùng thư viện google-cloud-compute, tạo máy với mô hình cấp phát SPOT và script khởi động (startup-script) tương tự. Muốn hỗ trợ thêm nhà cung cấp khác chỉ cần viết thêm một adapter.

Node Agent là tiến trình nhẹ viết bằng Python, chạy như dịch vụ systemd trên mỗi máy spot. Agent gửi heartbeat về control plane; theo dõi dịch vụ metadata của máy để phát hiện cảnh báo thu hồi; đồng bộ thư mục checkpoint lên object storage theo chu kỳ và ngay khi có cảnh báo; khi máy khởi động thì tải checkpoint về và chạy lệnh job của người dùng.

Lifecycle & Recovery nhận sự kiện thu hồi từ Agent, đồng thời coi việc mất heartbeat quá ngưỡng (ví dụ 90 giây) là dấu hiệu máy đã bị thu hồi mà Agent không kịp báo. Thành phần này điều phối quy trình khôi phục: chuyển máy sang RECOVERING, đưa nhóm máy vừa bị thu hồi vào danh sách loại trừ tạm thời, yêu cầu Instance Manager đặt lại máy, dọn tài nguyên cũ và ghi sự kiện để cập nhật thống kê rủi ro.

Cost & Metering ghi nhận khoảng thời gian chạy của từng instance nhân với giá tại thời điểm đó, tính chi phí on-demand tương đương, tổng hợp số lần thu hồi và thời gian gián đoạn.

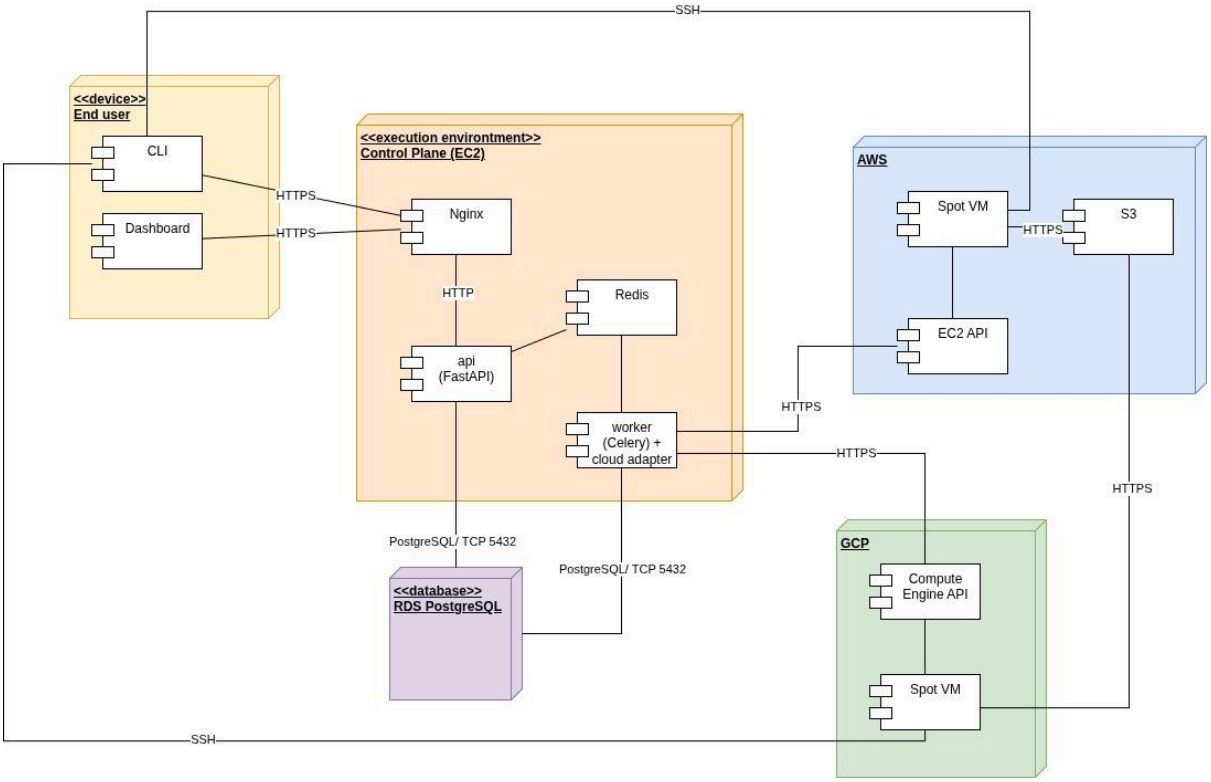
Tầng lưu trữ gồm cơ sở dữ liệu PostgreSQL (người dùng, khóa SSH, máy, instance, sự kiện, lịch sử giá, catalog) và object storage chứa checkpoint, tổ chức theo đường dẫn <người-dùng>/<máy>/.

2.2 Kiến trúc triển khai của hệ thống

Hình 2.2 miêu tả các thành phần được triển khai và các giao diện tương tác giữa chúng. Control plane được triển khai trên một máy ảo EC2, cơ sở dữ liệu sử dụng dịch vụ RDS PostgreSQL của AWS.

Hệ thống gồm bốn node triển khai. End user dùng CLI và Dashboard, kết nối tới control plane qua HTTPS. Control Plane là một máy EC2 chạy các container:

- Nginx là cổng vào duy nhất, chuyển tiếp yêu cầu tới api (FastAPI) qua HTTP nội bộ.
- Api đẩy các tác vụ chậm như tạo, xóa, khôi phục máy vào hàng đợi Redis.
- Worker (Celery + cloud adapter) lấy tác vụ từ Redis và gọi EC2 API của AWS hoặc Compute Engine API của GCP qua HTTPS để tạo, xóa Spot VM.



Hình 2.2: kiến trúc triển khai của spulti-cloud.

Dữ liệu được lưu trên RDS PostgreSQL, tách khỏi control plane; api và worker kết nối tới đây qua cổng PostgreSQL/ TCP 5432. Spot VM trên cả AWS và GCP đều đồng bộ checkpoint lên S3 qua HTTPS, nhờ đó dữ liệu đi theo máy khi chuyển cloud. Người dùng truy cập trực tiếp các Spot VM bằng SSH.

Máy spot được tạo từ Image Ubuntu 22.04 của các nhà cung cấp. Checkpoint được lưu trữ trên bucket S3 cho máy spot AWS và lưu trên GCS cho máy spot GCP.

Giao diện tương tác giữa các thành phần

Các thành phần chỉ giao tiếp với nhau qua các giao diện trong Bảng 2.1. Kết nối giữa các container đi trong mạng nội bộ Docker; mọi kết nối ra ngoài máy EC2 đều được mã hóa.

Bảng 2.1. Giao diện tương tác giữa các thành phần

Từ → Đến	Giao diện	Nội dung trao đổi
CLI, Dashboard → Nginx	HTTPS, REST API (JSON), token trong header	Đăng ký, đăng nhập, tạo, xem, xóa máy
Nginx → api	HTTP nội bộ	Chuyển tiếp nguyên request

api → Redis	Giao thức Redis	Đẩy tác vụ tạo, xóa, khôi phục máy; đọc trạng thái tác vụ
Redis → worker	Giao thức Redis (Celery)	Worker lấy tác vụ và ghi lại kết quả
api, worker → RDS	PostgreSQL qua TCP 5432, mã hóa TLS	Đọc, ghi người dùng, máy, instance, giá, sự kiện, chi phí
worker → AWS, GCP	HTTPS: EC2 API, Compute Engine API	Tạo, truy vấn, xóa Spot VM; lấy giá spot
Node Agent → api	HTTPS, REST API	Heartbeat, báo sự kiện thu hồi
Spot VM → S3, GCS	HTTPS	Đồng bộ và tải về checkpoint
Người dùng → Spot VM	SSH, cổng 22	Làm việc trực tiếp trên máy

Các thành phần triển khai

Nginx: Cổng vào duy nhất của control plane. Nginx giới hạn kích thước request và thời gian chờ phản hồi 30 giây, nên mọi thao tác chậm đều phải chuyển sang worker. Nginx chỉ khởi động khi api đã sẵn sàng.

api (FastAPI): Hiện thực tầng API & Xác thực và là điểm gọi vào tầng nghiệp vụ. api xử lý nhanh và trả lời ngay; việc gọi đám mây được đẩy vào hàng đợi. Toàn bộ cấu hình và bí mật được nạp từ biến môi trường, không nằm trong mã nguồn.

Redis: Hàng đợi tác vụ giữa api và worker. Redis ghi dữ liệu xuống đĩa mỗi giây và không tự xóa tác vụ khi đầy bộ nhớ, nên lệnh tạo, xóa máy không bị mất khi khởi động lại.

worker (Celery + Cloud Adapter): Chạy các tác vụ chậm; tạo một Spot VM có thể mất vài phút. Chỉ worker được cấp quyền gọi API đám mây: AWS qua IAM role của máy EC2, GCP qua khóa service account. AWS Adapter và GCP Adapter cùng một giao diện tạo, truy vấn, xóa máy; mỗi yêu cầu mang một mã duy nhất để gửi lại sau lỗi mạng không tạo trùng máy.

RDS PostgreSQL. Cơ sở dữ liệu tách khỏi máy EC2, nên có thể dựng lại control plane mà không mất dữ liệu. RDS nằm cùng VPC với EC2, không có địa chỉ công khai và chỉ nhận kết nối từ control plane.

Spot VM và Node Agent. Spot VM chạy Ubuntu 22.04, có IP công khai để người dùng SSH vào. Script khởi động cài khóa SSH của người dùng và Node Agent; agent gửi heartbeat, bắt cảnh báo thu hồi và đồng bộ checkpoint. Ổ đĩa của máy bị xóa khi máy bị thu hồi, chỉ checkpoint được giữ lại.

Object storage (S3, GCS). Lưu checkpoint bên ngoài Spot VM, theo từng người dùng và từng máy, để máy thay thế tải về và chạy tiếp.

CHƯƠNG 3: KẾ HOẠCH TRIỂN KHAI VÀ ƯỚC TÍNH NỖ LỰC

Bảng 3.1. Kế hoạch hạng mục công việc

STT	Hạng mục	Nội dung chính	Thành phần kiến trúc liên quan	Phụ trách
1	Chuẩn bị hạ tầng đám mây	Tạo tài khoản, phân quyền, mạng, bucket S3/GCS và CSDL RDS	AWS (EC2 API, S3), GCP (Compute Engine API, GCS), RDS PostgreSQL	TV1
2	Dựng Control Plane	Chạy Nginx, FastAPI, Redis, Celery bằng Docker trên EC2	Control Plane (EC2): Nginx, api, Redis, worker	TV2
3	Thiết kế CSDL	Thiết kế bảng lưu người dùng, máy, sự kiện, giá, chi phí	RDS PostgreSQL	TV3
4	API và xác thực	Đăng ký, đăng nhập, quản lý khóa SSH và hạn mức máy	Nginx, api (FastAPI)	TV1
5	Catalog	Quy đổi loại máy AWS/GCP về cấu hình chung (vCPU, RAM, GPU)	api (FastAPI), RDS PostgreSQL	TV3
6	Market Data Collector	Thu thập định kỳ giá spot, giá on-demand và rủi ro thu hồi	worker (Celery) + cloud adapter, Redis	TV3

STT	Hạng mục	Nội dung chính	Thành phần kiến trúc liên quan	Phụ trách
7	AWS Adapter	Tạo, xóa, theo dõi máy EC2 Spot	worker + cloud adapter, EC2 API	TV2
8	GCP Adapter	Tạo, xóa, theo dõi Spot VM của GCP	worker + cloud adapter, Compute Engine API	TV2
9	Placement Engine	Chấm điểm và chọn máy tốt nhất theo 3 chế độ tối ưu	api (FastAPI)	TV1
10	Instance Manager	Quản lý vòng đời máy và trạng thái từ lúc tạo đến lúc xóa	api (FastAPI), Redis, worker (Celery)	TV1
11	Node Agent	Chạy trên máy spot: báo trạng thái, đồng bộ checkpoint, chạy job	Spot VM (AWS, GCP), S3, GCS	TV3
12	Phát hiện thu hồi	Bắt cảnh báo thu hồi, lưu checkpoint lần cuối và báo về nền tảng	Spot VM (AWS, GCP)	TV2
13	Lifecycle & Recovery	Tạo máy thay thế, khôi phục checkpoint và chạy lại job	worker (Celery) + cloud adapter, Spot VM, S3, GCS	TV1
14	Cost & Metering	Tính chi phí thực tế và phần trăm tiết kiệm	api (FastAPI), RDS PostgreSQL	TV3
15	CLI spulti	Các lệnh tạo, xem, SSH, xem log, xem chi phí, xóa máy	End user: CLI	TV1
16	Web Dashboard	Giao diện tạo máy, danh sách máy và chi tiết máy	End user: Dashboard	TV2

TV1: Vũ Trí Sơn, TV2: Lê Thành Long, TV3: Lại Tùng Lâm.

Bảng 3.2. Trạng thái các thành phần triển khai

Thành phần	Trạng thái	Còn thiếu
Nginx	Hoàn thiện	Optional: HTTPS, giới hạn tần suất đăng nhập

api	Một phần	API key, khóa SSH, quản lý máy, hạn mức số máy
Redis	Hoàn thiện	Không
worker	Một phần	Tác vụ tạo, xóa, khôi phục máy; thu thập giá định kỳ
RDS PostgreSQL	Hoàn thiện	Optional: Công cụ cập nhật lược đồ (migration)
Spot VM	Một phần	Script khởi động đầy đủ, khóa SSH của từng người dùng
Node Agent	Chưa làm	Toàn bộ
Object storage	Chưa làm	Tạo bucket, đồng bộ checkpoint
CLI, Dashboard	Chưa làm	Toàn bộ

Ước tính nỗ lực:

Vũ Trí Sơn: 30%

Lại Tùng Lâm: 35%

Lê Thành Long: 35%